



XAI Learning Recommendation System — Master Plan

Merged from `expanded-mlops-system-design` + `aws-mlops-system-design` .

Single source of truth covering architecture, gaps, implementation phases, AWS provisioning, and operational runbook.

1. Architecture Overview

1.1 ML Pipeline (Two-Stage)

Stage 1 – Engagement Encoding

ImplicitAggregator → 20 signals (15 implicit + 5 explicit)

→ EngagementAutoencoder → 3 latent features

Stage 2 – Risk Prediction

12 OULAD features + 3 latent = 15 features → GBM (primary) / RF / LSTM

1.2 Recommendation Engine

Student path: `POST /recommend/student`

→ LambdaMART student_ranker (LGBMRanker)

→ RankerExplainer (TreeSHAP + Anchors + Interactions + Causal + Stability)

→ FairnessAuditor (demographic parity across gender/age/disability)

Instructor path: `POST /instructor/recommend/{learner_id}`

→ LambdaMART instructor_ranker (34-feature schema)

→ Full RankerExplainer stack

1.3 XAI Stack (Parallel at /explain time)

Explainer	Output
TreeSHAP / DeepSHAP / Captum IG	Feature attribution
Archipelago	Feature interaction pairs
Anchors	IF-THEN rule with precision/coverage
DiCE	Counterfactuals (immutable feature locks)
Prototypes	k-NN similar learners
MAPIE	Conformal prediction uncertainty
CausalAnnotator	DoWhy DAG — causal vs correlational
TrustScorer	$0.40 \times \text{fidelity} + 0.35 \times \text{stability} + 0.25 \times \text{completeness}$
LLM Narrator	Groq — narrates pre-computed XAI payloads only

1.4 MLOps Feedback Loop

Users → /events (implicit) + /feedback (explicit) → PostgreSQL

↓

RetrainPipeline (weekly cron or manual):

1. ImplicitAggregator: EventStore → 20 per-learner engagement signals
2. EngagementAutoencoder: 20 → 3 latent features
3. Augment: OULAD_12 + latent_3 = 15 features
4. Retrain GBM + RF, validate: AUC drop ≤ 0.02 , Brier rise ≤ 0.03 , SHAP fidelity ≤ 0.05
5. Save to disk → upload to S3

RecommendationRetrainPipeline (same trigger):

1. Same ImplicitAggregator signals
2. Enrich recommendation training data with learned_implicit_* columns
3. Retrain student_ranker + instructor_ranker (LambdaMART)
4. Regenerate precomputed top-K CSVs
5. Save to disk → upload to S3

Hot-Reload → POST /mlops/reload-all:

- S3 download → atomic swap of ALL 12+ components
- Canary mode supported (fractional traffic routing)

1.5 Drift Monitoring

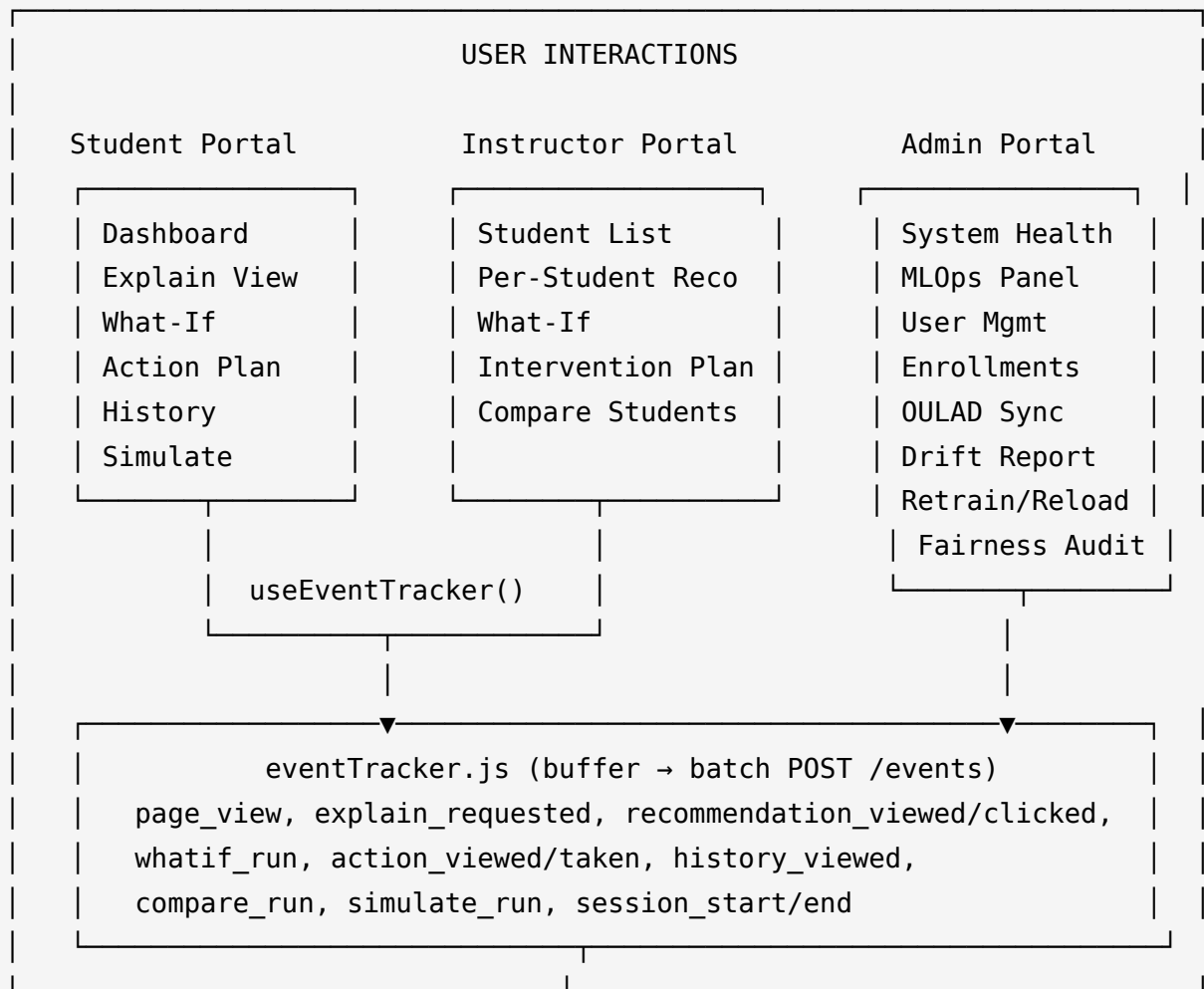
- **Evidently DriftMonitor**: recent prediction inputs vs training distribution
- **ExplanationDriftDetector**: JSD on |SHAP| softmax distributions
- **ConceptDriftProxy**: calibration gap between risk scores and follow rates
- **Slack webhook** alert when drift_share $\geq 20\%$
- **Trend tracking**: last 5 drift reports

1.6 CI/CD Pipelines

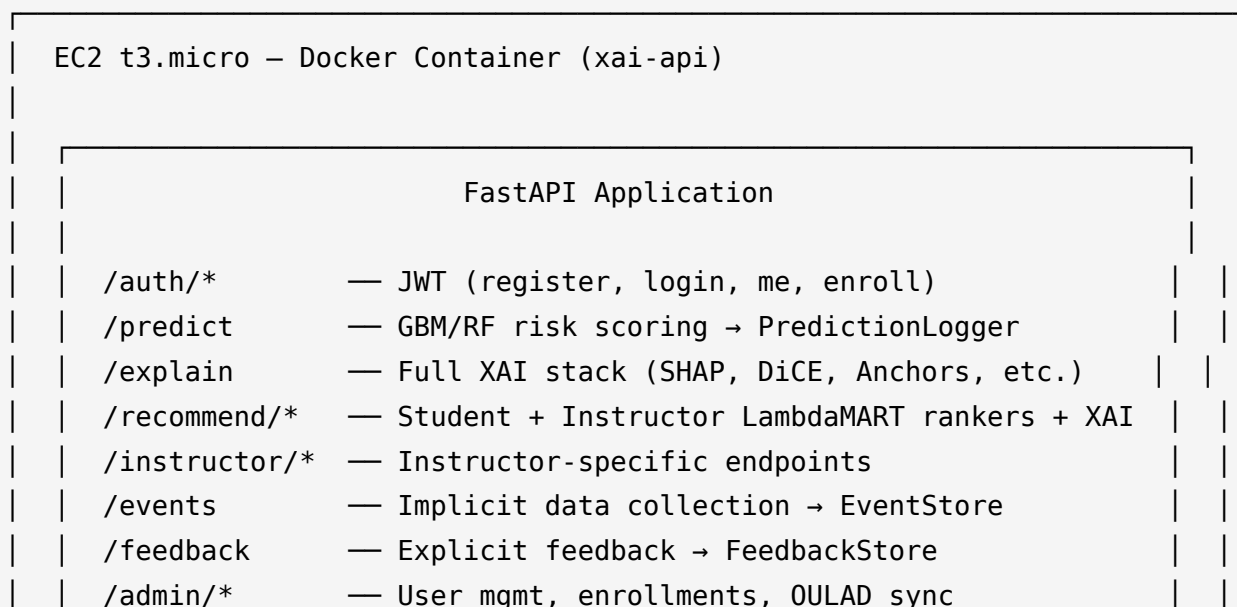
Workflow	Trigger	What It Does
ci.yml	Push/PR to main	Ruff lint + pytest (Postgres service container) + frontend build check + smoke tests
deploy.yml	Push to main	Docker → ECR → SSH EC2 → Alembic migrate →

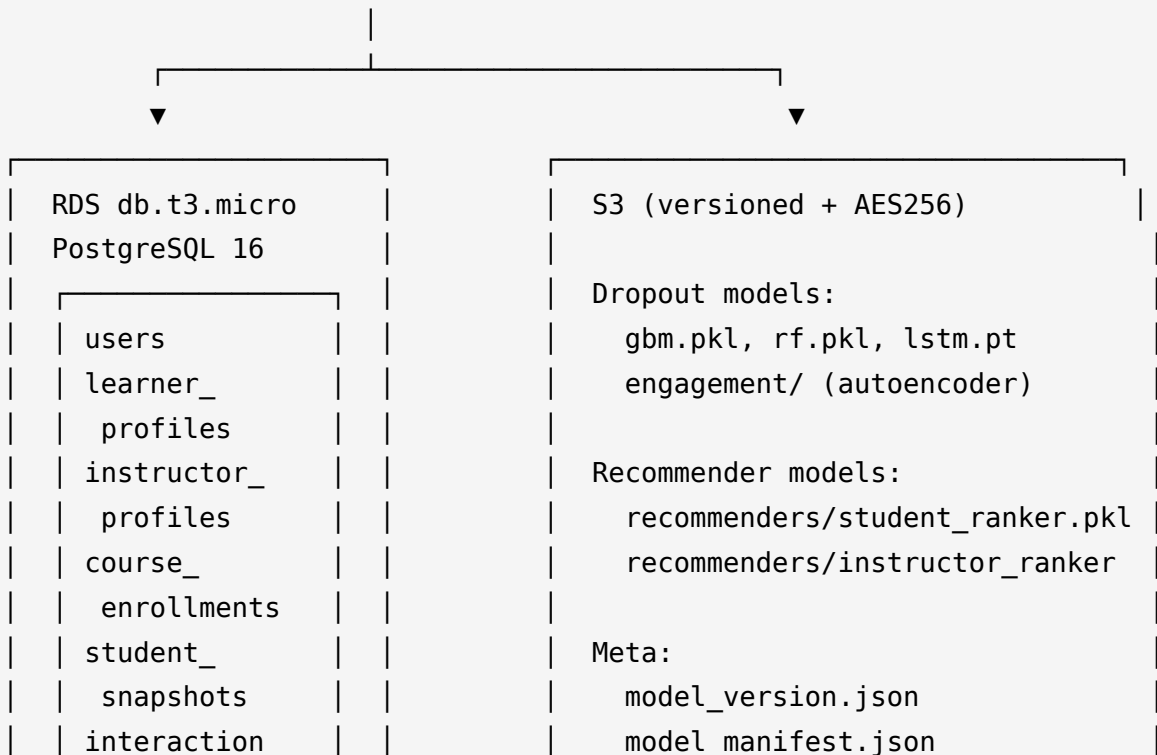
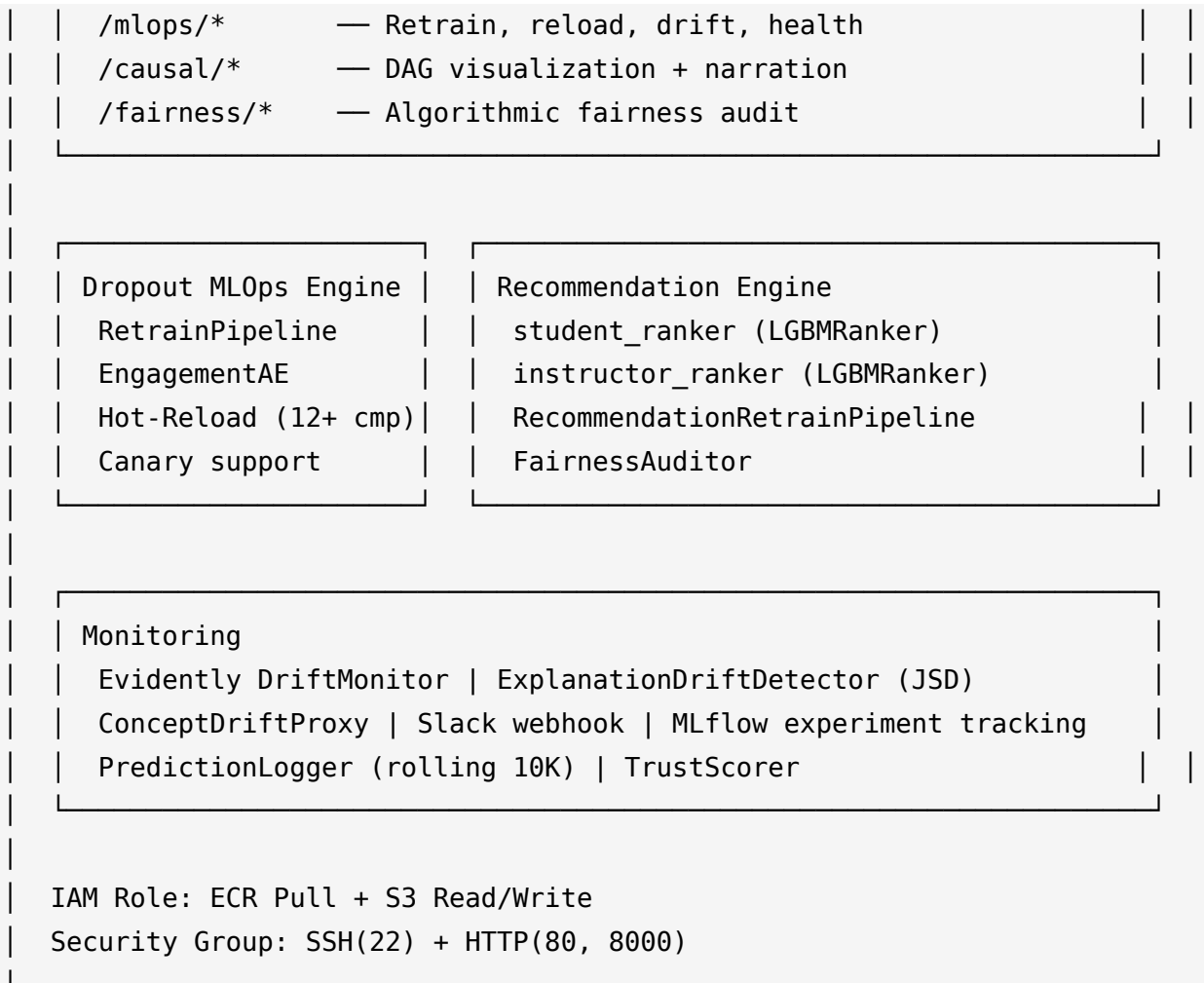
Workflow	Trigger	What It Does
		systemd restart → health check → rollback on failure
<code>retrain.yml</code>	Weekly Sun 02:00 UTC + manual	POST /mlops/retrain-all → POST /mlops/reload-all → health check
<code>terraform.yml</code>	Changes to <code>infra/**</code>	Plan on PR (post as comment) → Apply on merge

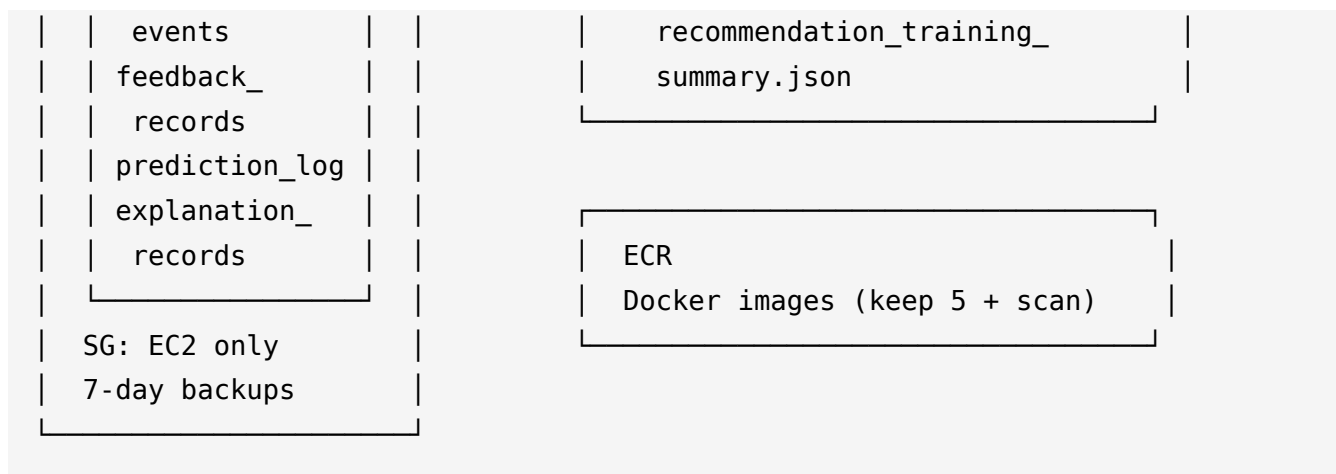
2. Full Data Flow Diagram



HTTP :8000







3. Security Model

Layer	Mechanism
Network	RDS private (SG allows only EC2 SG), SSH restricted to admin IP
Auth	JWT HS256, 24h expiry — student / instructor / admin roles
MLOps protection	<code>/mlops/retrain</code> + <code>/mlops/reload</code> require admin JWT OR <code>X-MLOPS-Token</code> header
Secrets	GitHub Secrets → EC2 <code>/opt/xai/.env</code> (chmod 600) — never in Terraform state
S3	Private, blocked public access, AES256 SSE
ECR	Scan-on-push for vulnerabilities
Docker	Non-root user (<code>xaiuser</code>), health checks
DB	Alembic-managed migrations, connection string via env only

Cross-Portal Visibility

Actor	Can See	Mechanism
Student	Own dashboard, risk, explanations,	JWT learner_id → profile filter

Actor	Can See	Mechanism
	recommendations, history	
Instructor	Enrolled students' risk, per-student recommendations, what-if	CourseEnrollment join, InstructorProfile filter
Admin	All users, all enrollments, system health, MLOps, drift, fairness	role="admin", no enrollment filter

4. Implementation Gap Analysis & Status

✓ Complete (No Changes Needed)

Component	Files
Auth (JWT, bcrypt, 24h)	auth/auth.py , auth/router.py , auth/models.py , auth/schemas.py
Role guards	get_current_user , get_current_active_student , get_current_instructor , get_current_admin
Student recommendation + XAI	POST /recommend/student + /recommend/student/explain
Instructor recommendation + XAI	POST /instructor/recommend/{learner_id} + explain
Recommendation retraining pipeline	RecommendationRetrainPipeline
Event tracking backend	POST /events → EventStore (PostgreSQL)
Event tracker	eventTracker.js (buffer, 10s flush, beforeunload),

Component	Files
frontend service	<code>useEventTracker.js</code>
Feedback collection	<code>POST /feedback</code> → FeedbackStore
All frontend pages	Login, Register, Admin/Instructor/Student Dashboards, WhatIf, History, ActionPlan, Compare, Simulate
Dropout model pipeline	GBM/RF/LSTM + EngagementAutoencoder + RetrainPipeline + validation gates
Drift monitoring	Evidently + JSD + ConceptDriftProxy + Slack alerts
MLOps endpoints	<code>/mlops/retrain</code> , <code>/mlops/retrain-all</code> , <code>/mlops/reload</code> , <code>/mlops/reload-all</code> , <code>/mlops/drift-report</code> , <code>/mlops/health</code>
Demo seeder	<code>_seed_demo_users()</code> seeds 3 core + 6 roster students + enrollments + snapshots
Admin enrollment endpoints	<code>GET/POST/DELETE /admin/enrollments</code> , <code>GET /admin/instructors</code>
Admin user sync endpoints	<code>POST /admin/sync-oulad</code> , <code>GET /admin/students</code> , <code>POST /admin/users/{id}/activate</code> , <code>POST /admin/users/{id}/deactivate</code>
Admin bulk enrollment	<code>POST /admin/enrollments/bulk</code>
S3 recommender sync	<code>s3_loader.py</code> — <code>recommenders/</code> dir upload/download, <code>recommendation_training_summary.json</code>
Navbar admin routing	Admin role sees Instructor View + Admin Dashboard links
Sidebar	Sign Out button, expanded admin links (What-If, Simulate, Compare, XAI Tools)
<code>/admin</code> route	Redirects to <code>/dashboard/admin</code>

Component	Files
Admin Dashboard UI	OULAD Sync button, User Management table with risk badges + activate/deactivate
Event type constants	<code>frontend/src/constants/eventTypes.js</code>
Page event tracking	<code>useEventTracker</code> + <code>PAGE_VIEW</code> wired in all 11 pages; <code>WHATIF_RUN</code> on mutation
learner_id propagation	<code>setLearnerId</code> called on login/logout/signup in AuthContext
Cold-start StudentSnapshot	Created automatically on student registration
learner_id auto-generation	Backend generates <code>learner_{uuid8}</code> if student registers without providing one

✖ Remaining Gaps

#	Gap	Severity
G1	Precomputed CSVs (<code>student_topk.csv</code> , <code>instructor_topk.csv</code>) not yet included in S3 upload/download	MEDIUM
G2	AWS infrastructure not yet provisioned (Terraform never applied)	BLOCKER for deploy
G3	GitHub Secrets not yet configured	BLOCKER for CI/CD
G4	Initial model upload to S3 not done	BLOCKER for deploy

5. Implementation Phases

Phase A — S3 Precomputed CSV Sync (G1) ~15 min

Add `data/recommendations/precomputed/student_topk.csv` and `instructor_topk.csv` to `s3_loader.py` upload/download.

File: `backend/app/model/s3_loader.py`

```
# In _DATA_FILES (new list) or alongside _MODEL_FILES:
_PRECOMPUTED_FILES = [
    ("data/recommendations/precomputed/student_topk.csv", "precomputed/student_topk.csv"),
    ("data/recommendations/precomputed/instructor_topk.csv", "precomputed/instructor_topk.csv"),
]
```

Phase B — AWS Bootstrap (G2, G3, G4) ~30 min total

B.1 One-Time Manual Setup (~15 min)

```
# 1. Create IAM user xai-ci-cd with these policies:
#   AmazonEC2FullAccess, AmazonRDSFullAccess, AmazonS3FullAccess,
#   AmazonEC2ContainerRegistryFullAccess, AmazonDynamoDBFullAccess
# Save: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY

# 2. Create Terraform state backend
aws s3 mb s3://xai-rec-tf-state --region ap-south-1
aws dynamodb create-table \
  --table-name xai-rec-tf-locks \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region ap-south-1

# 3. Generate SSH key pair
ssh-keygen -t rsa -b 4096 -f ~/.ssh/xai-rec-key -N ""

# 4. Create tfvars
cp infra/terraform.tfvars.example infra/terraform.tfvars
# Set: db_password, ssh_public_key_path, allowed_ssh_cidr (your IP)
```

B.2 Provision Infrastructure (~10 min)

```
cd infra
terraform init      # Connects to S3 state backend
terraform validate
terraform plan
terraform apply     # Creates: EC2 + RDS + ECR + S3 + VPC/SGs
```

Save these outputs for GitHub Secrets:

Output	Secret Name
ec2_public_ip	EC2_HOST

Output	Secret Name
<code>ecr_repo_url</code>	<code>ECR_REPO_URL</code>
<code>database_url</code>	<code>DATABASE_URL</code>
<code>s3_bucket_name</code>	<code>AWS_S3_BUCKET</code>

B.3 Configure GitHub Secrets (~5 min)

GitHub → Repo Settings → Secrets and variables → Actions:

Secret	Value
<code>AWS_ACCESS_KEY_ID</code>	From B.1
<code>AWS_SECRET_ACCESS_KEY</code>	From B.1
<code>AWS_REGION</code>	<code>ap-south-1</code>
<code>ECR_REPO_URL</code>	From <code>terraform output ecr_repo_url</code>
<code>EC2_HOST</code>	From <code>terraform output ec2_public_ip</code>
<code>EC2_SSH_KEY</code>	Contents of <code>~/.ssh/xai-rec-key</code> (private key)
<code>DATABASE_URL</code>	From <code>terraform output -raw database_url</code>
<code>JWT_SECRET_KEY</code>	<code>python -c "import secrets; print(secrets.token_hex(32))"</code>
<code>GROQ_API_KEY</code>	From https://console.groq.com
<code>AWS_S3_BUCKET</code>	From <code>terraform output s3_bucket_name</code>
<code>DB_PASSWORD</code>	Same as in <code>terraform.tfvars</code>
<code>MLOPS_AUTOMATION_TOKEN</code>	<code>python -c "import secrets; print(secrets.token_hex(32))"</code>

Also create a GitHub Environment named `production` (Settings → Environments).

B.4 Initial Model Upload to S3 (~2 min)

```
export AWS_S3_BUCKET=$(terraform -chdir=infra output -raw s3_bucket_name)
make models-upload
# Uploads: gbm.pkl, rf.pkl, lstm.pt, engagement/, recommenders/,
#          model_version.json, model_manifest.json, recommendation_training_summary.json
```

B.5 First Deployment (~10 min)

```
git add . && git commit -m "deploy: initial production release"
git push origin main
```

`deploy.yml` runs automatically:

1. Multi-stage Docker build (React frontend + Python backend)
2. Push to ECR (tagged with commit SHA + timestamp)
3. SSH to EC2 → write secrets to `/opt/xai/.env`
4. Pull image by digest (immutable)
5. `alembic upgrade head` (creates all DB tables)
6. Restart `xai-api` systemd service
7. Health check loop (up to 2 min)
8. On failure: automatic rollback to previous image

B.6 Verify Deployment (~5 min)

```
EC2_IP=$(terraform -chdir=infra output -raw ec2_public_ip)

curl -sf http://$EC2_IP:8000/health | python -m json.tool

curl -X POST http://$EC2_IP:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"login_frequency_weekly":3,"avg_session_duration_min":45,"forum_posts_count":2,
      "video_completion_rate":0.6,"quiz_avg_score":55,"quiz_completion_rate":0.7,
      "assignment_submission_rate":0.65,"days_since_last_activity":7,
      "prior_course_completions":1,"current_week_in_course":6,
      "missed_deadlines_count":2,"help_requests_count":1,
      "engagement_latent_1":0,"engagement_latent_2":0,"engagement_latent_3":0}'

echo "Swagger: http://$EC2_IP:8000/docs"
echo "Frontend: http://$EC2_IP:8000/login"
```

6. Operational Runbook

6.1 Weekly Retrain (Automatic)

Configured in `retrain.yml` — runs every **Sunday 02:00 UTC** or manually via GitHub Actions UI:

1. `POST /mlops/retrain-all` (with `X-MLOPS-Token`)
2. On success: `POST /mlops/reload-all`
3. Health check confirms live

6.2 Rollback Scenarios

Scenario	Mechanism
Deploy fails health check	<code>deploy.yml</code> reverts to previous Docker image digest
Retrain fails validation	<code>RetrainPipeline</code> returns <code>success: false</code> , model

Scenario	Mechanism
gates	unchanged
Hot-reload fails	Staging dict discarded, AppState untouched
Canary shows issues	POST /mlops/reload?canary_fraction=0.0 aborts canary
Infra drift	Terraform state in S3; terraform plan surfaces drift

6.3 Admin Workflows

Task	Action
Bulk-create users from OULAD	Admin Dashboard → Sync OULAD Users button
View / activate / deactivate users	Admin Dashboard → User Management table
Add enrollment	Admin Dashboard → Enrollment Management → Add Enrollment form
Bulk enroll	POST /admin/enrollments/bulk (API or future bulk UI)
Trigger retrain	Admin Dashboard → MLOps Controls → Run Retrain
Hot-reload model	Admin Dashboard → MLOps Controls → Hot Reload
Check drift	Admin Dashboard → Drift Report section
Fairness audit	Admin Dashboard → Fairness Monitor → Run Fairness Check

6.4 Auth Flows

Login:

```
POST /auth/login { username, password }
→ bcrypt verify → JWT { sub: user_id, role, learner_id, exp: 24h }
→ Frontend: localStorage(ll_token, ll_user) + setLearnerId(learner_id)
→ Redirect: student→/student, instructor→/instructor, admin→/dashboard/admin
```

Registration (Student):

```
POST /auth/register { username, email, password, role="student", ... }
→ Creates User + LearnerProfile
→ Auto-generates learner_id if not provided (learner_{uuid8})
→ Creates cold-start StudentSnapshot (enables dashboard immediately)
→ Returns { id, username, email, role, learner_id }
```

Admin OULAD Sync:

```
POST /admin/sync-oulad (admin JWT required)
→ Reads data/oulad_seed_credentials.json
→ For each entry: create User + Profile (skip if username exists)
→ For students: create StudentSnapshot with OULAD features
→ Returns { created_users, skipped_existing, snapshots_created, errors }
```

7. Cost Estimate (AWS Free Tier)

Resource	Free Tier	Our Usage	Notes
EC2 t3.micro	750 hrs/month (12 mo)	1 instance 24/7	Within free tier
RDS db.t3.micro	750 hrs/month (12 mo)	1 instance, 20GB	Within free tier
S3	5GB, 20K GET, 2K PUT	~300MB models + CSVs	Within free tier
ECR	500MB storage	~1.2GB (5 images)	~\$0.05/month

Resource	Free Tier	Our Usage	Notes
			overage
DynamoDB	25GB + 25 WCU/RCU	TF state lock only	Within free tier
Data transfer	100GB out/month	Minimal	Within free tier

Estimated monthly cost: ~\$0.05 (ECR overage only).

8. File Reference

Backend

File	Purpose
<code>backend/app/main.py</code>	All FastAPI endpoints (3500+ lines)
<code>backend/app/auth/router.py</code>	register, login, me, enroll — auto learner_id + cold-start snapshot
<code>backend/app/auth/models.py</code>	User, LearnerProfile, InstructorProfile, CourseEnrollment, StudentSnapshot
<code>backend/app/model/s3_loader.py</code>	S3 upload/download — dropout models + engagement/ + recommenders/
<code>backend/app/model/retrain_pipeline.py</code>	Dropout model retraining
<code>backend/app/recommender/train_recommenders.py</code>	LambdaMART ranker training
<code>backend/app/recommender/retrain_pipeline.py</code>	Recommendation retraining pipeline

File	Purpose
<code>backend/app/tracker/event_store.py</code>	EventStore (PostgreSQL)
<code>backend/app/tracker/drift_detector.py</code>	JSD-based explanation drift
<code>backend/app/mlops/drift_monitor.py</code>	Evidently drift monitor
<code>backend/app/narrator/llm_narrator.py</code>	Groq narration + rule-based fallback
<code>alembic/versions/</code>	DB schema migrations
<code>infra/</code>	Terraform (EC2, RDS, S3, ECR, VPC)
<code>.github/workflows/</code>	ci.yml, deploy.yml, retrain.yml, terraform.yml

Frontend

File	Purpose
<code>frontend/src/constants/eventTypes.js</code>	Canonical event type constants
<code>frontend/src/context/AuthContext.jsx</code>	Auth state + setLearnerId on login/logout
<code>frontend/src/api/recommend.js</code>	API client — recommend, admin, sync-oulad, bulk-enroll
<code>frontend/src/api/dropout.js</code>	API client — predict, MLOps, health
<code>frontend/src/components/Navbar.jsx</code>	Role-aware nav (student/instructor/admin links)
<code>frontend/src/components/Sidebar.jsx</code>	Role-aware sidebar, Sign Out button
<code>frontend/src/pages/AdminDashboard.jsx</code>	System health, MLOps, enrollment, user management, OULAD sync
<code>frontend/src/pages/StudentDashboard.jsx</code>	Risk score, recommendations,

File	Purpose
	page_view + reco events
frontend/src/pages/InstructorDashboard.jsx	Student roster, recommendations
frontend/src/pages/WhatIfExplorer.jsx	What-if analysis + whatif_run event
frontend/src/pages/SimulatePage.jsx	Monte Carlo simulation + simulate_run event
frontend/src/pages/ComparePage.jsx	Model comparison + compare_run event
frontend/src/pages/HistoryPage.jsx	Explanation history + history_viewed event
frontend/src/pages/ActionPlan.jsx	Prioritised actions + action_viewed event
frontend/src/App.jsx	Routes — /admin → /dashboard/ admin redirect

9. Implicit Data → MLOps: End-to-End Trace

1. Student clicks "View Explanation" on StudentView
 - `useEventTracker.track(EXPLAIN_REQUESTED, { target: learner_id })`
 - Buffered in `eventTracker.js`
2. Buffer hits 50 events OR 10s timer fires (or beforeunload)
 - `POST /events { events: [...50 events] }`
 - EventStore writes to PostgreSQL `interaction_events`
3. Sunday 02:00 UTC – `retrain.yml` fires
 - `POST /mlops/retrain-all (X-MLOPS-Token)`
 - RetrainPipeline:
 - a. ImplicitAggregator: EventStore → 20 engagement signals per learner
 - b. EngagementAutoencoder: 20 → 3 latent features
 - c. Augment: OULAD_12 + latent_3 = 15 features
 - d. Retrain GBM + RF, validate gates (AUC/Brier/SHAP)
 - RecommendationRetrainPipeline:
 - a. Same signals → enrich recommendation training data
 - b. Retrain student_ranker + instructor_ranker (LambdaMART)
 - c. Regenerate precomputed top-K CSVs
 - Both save to disk → upload to S3
4. `POST /mlops/reload-all`
 - S3 download → atomic swap of ALL 12+ components
 - Health check confirms live
 - Zero downtime (canary mode optional)

Last updated: 2025-07-17. All Phase 1–3 implementation tasks complete. Remaining blocker: AWS provisioning (Phase B).